

Introduction to Components

The component-based approach is a popular methodology in web development, used in all modern JavaScript frameworks. It allows you to break an interface into independent functional blocks, and each block into even smaller components. For example, any form can be divided into input fields, buttons, checkboxes, and other elements.

If components possess a basic set of properties, their behavior becomes predictable. This greatly simplifies development.

The following can be highlighted as basic properties:

- Template support;
- Storage and updating of properties (state) of the component;
- Lifecycle;
- Event handling.

We will not use ready-made frameworks — instead, we will implement our own, under the code name "React on a shoestring." In this topic, we will create a base class for our components and make them reactive.

You already know how to bootstrap projects with Vite and TypeScript. That is exactly what we need for this topic. Move on to the next lesson once you have set up your local project!

Block

Let's start with the "foundation." Literally — we will create an abstract base class.

All other components will be created from our base class. Since we do not know in advance what properties child components will have, we will use a generic type:

```
| abstract class Block<Props extends object> {}
```

Template String and Compilation

Any component consists of a set of DOM elements assembled in a specific structure. Instead of creating many base classes for each element structure, it is simpler to add template support. Handlebars, which you are already familiar with, will help us with this.

Let's add a property to the base class for storing the template string. We will make this property protected and abstract: the property is part of the overall structure of the abstract base class, but will have different values for each subclass:

```
abstract class Block<Props extends object> {  
  /** Template string from which the component's markup will be generated */  
  /** This is a required property for all components, so we declare it protected  
  and abstract */  
  protected abstract template: string;  
  
  /** Template properties: button text, input placeholder, etc. */  
  protected props = {} as Props;  
  
  /** DOM element created from the template string */  
  private domElement: Element | null = null;  
}
```

For convenient interaction with the component's properties, it is better to store them in the class instance. The component's properties will be passed from the outside — from the parent. Let's add them as a constructor parameter and save them to a class property:

```
/** If the component has no properties, we set an empty object */  
constructor(props: Props = {} as Props) {  
  this.props = props;  
}
```

Next, we will need a method to transform the template string into real DOM elements. The process of converting a template string into valid HTML markup is called compilation. This logic will be common to all components, so it is convenient to extract it into a base class method. Moreover, the method should be made private. Preventing overriding of this method in subclasses adds flexibility in the future: it will allow easily changing the template engine or other internal logic without changing the public interface.

Let's add a private method compile. We will get valid HTML from the template string using Handlebars. For now, this is still just a string and needs to be turned into real DOM elements. We'll use `document.createElement` to create a template element. The element created this way will be stored in memory and will not affect what is already on screen.

Using `innerHTML`, we will add the compiled string to the created element. The template element's content property returns a `DocumentFragment` — a minimal document object with no parent. For simplicity, we take the first child element `firstElementChild`. This way, we ensure that our component template strings can only have one root element; the rest will be ignored:

```
private compile(): Element | null {  
  const html = Handlebars.compile(this.template)(this.props);  
  const templateElement = document.createElement("template");  
  templateElement.innerHTML = html;  
}
```

```
    return templateElement.content.firstElementChild;
}
```

Inserting and Replacing an Element in the DOM Tree

Now let's add a method that saves the created element in the class instance and replaces it in the DOM tree if it has already been added. We declare it protected, because it will be extended with additional logic later:

```
protected render() {
  const fragment = this.compile();
  /** If the element already existed, update it using the existing reference */
  if (this.domElement && fragment) {
    this.domElement.replaceWith(fragment);
  }
  this.domElement = fragment;
}
```

Finally, let's write a method that will return the element:

```
public element(): Element | null {
  if (!this.domElement) {
    this.render();
  }
  return this.domElement;
}
```

The First "Real" Component

Here is the full code for the Block class:

```
import Handlebars from 'handlebars';

export default abstract class Block<Props extends object> {
  protected abstract template: string;
  protected props = {} as Props;
  private domElement: Element | null = null;

  constructor(props: Props = {} as Props) {
    this.props = props;
  }

  public element(): Element | null {
    if (!this.domElement) {
      this.render();
    }
    return this.domElement;
  }

  protected render() {
    const fragment = this.compile();
    if (this.domElement && fragment) {
```

```

        this.domElement.replaceWith(fragment);
    }
    this.domElement = fragment;
}

private compile(): Element | null {
    const html = Handlebars.compile(this.template)(this.props);
    const templateElement = document.createElement("template");
    templateElement.innerHTML = html;
    return templateElement.content.firstElementChild;
}
}

```

This is a minimal implementation that already allows you to create your own components. Yes, without reactivity and event handling, but we will add those. For now, let's write our first component — a form with two input fields and a submit button. All files are at the same level, without subfolders or project organization.

Let's describe the component in a `Form.ts` file. In the `main.ts` file, we create an instance of the form and render it into the body:

```

import Block from './Block'

export default class Form extends Block {
    protected template = `
        <form>
            <input type="text" placeholder="Login" ref="login">
            <input type="password" placeholder="Password" ref="password">
            <button>Submit</button>
        </form>
    `;
}

import Form from './Form';

const form = new Form();
const FormElement = form.element();
document.body.appendChild(FormElement);

```

In this lesson, we laid the foundation for our "React on a shoestring." We can create our own component and even display it on the page. We cannot do much more with it yet, but that is only for now. In the following lessons, we will add the necessary functionality. We will start with reactivity — the ability to update the view when input data changes.

Reactive

We worked on the foundation — we wrote a base class for components. Time to talk about reactivity.

Reactivity (not to be confused with reactive programming) means the ability of a system to track changes in input data, automatically update its state, and output the updated results. For a component, this means that when input data changes, it must update its representation in the DOM tree.

Let's add a method for updating the component's properties:

```
public setProps(props: Partial<Props>) {  
  /** Merge the updated properties */  
  this.props = { ...this.props, ...props };  
  /** Call the render method, updating the representation in the DOM tree */  
  this.render();  
}
```

When the setProps method is called, the render method will be invoked: the component's state will be updated, along with its visual representation.

Now the project code looks like this:

```
import Handlebars from 'handlebars';  
  
export default abstract class Block<Props extends object> {  
  protected abstract template: string;  
  protected props = {} as Props;  
  private domElement: Element | null = null;  
  
  constructor(props: Props = {} as Props) {  
    this.props = props;  
  }  
  
  public element(): Element | null {  
    if (!this.domElement) {  
      this.render();  
    }  
    return this.domElement;  
  }  
  
  protected render() {  
    const fragment = this.compile();  
    if (this.domElement && fragment) {  
      this.domElement.replaceWith(fragment);  
    }  
    this.domElement = fragment;  
  }  
  
  private compile(): Element | null {  
    const html = Handlebars.compile(this.template)(this.props);  
    const templateElement = document.createElement("template");  
    templateElement.innerHTML = html;  
    return templateElement.content.firstChild;  
  }  
}
```

```

    public setProps(props: Partial<Props>) {
      this.props = { ...this.props, ...props };
      this.render();
    }
  }
}

import Block from './Block';

export default class Form extends Block {
  protected template = `
    <form>
      <input type="text" placeholder="Login" ref="login">
      <input type="password" placeholder="Password" ref="password">
      <button>{{ buttonName }}</button>
    </form>
  `;
}

import Form from './Form';

const form = new Form({ buttonName: "Initial state" });
const FormElement = form.element();
document.body.appendChild(FormElement);
form.setProps({ buttonName: "Click!" });

```

In this lesson, we taught components to update their representation when input data changes, and confirmed that it is not difficult. This is a significant improvement, but there is still a lot more ahead.

Lifecycle

The next stage in the evolution of "React on a shoestring" is the lifecycle. In this lesson, you will learn what it is and why it is needed.

The lifecycle is the stages a component goes through from appearing on the page to being removed. The basic lifecycle consists of three stages:

- Creating an instance of the component.
- Adding the component to the DOM tree.
- Removing the component.

The Three Lifecycle Stages

Here they are — from left to right:

- **Render.** This is where preliminary preparation happens: compiling the template, creating the actual DOM elements.

- Commit, or mounting. At this stage, the component is added to the DOM tree and the `componentDidMount` signal fires. In its handler, you can make server requests, work with events, and access component dimensions.
- Unmounting. The component is removed from the DOM tree, and the `componentWillUnmount` signal is triggered. At this stage, cleanup logic is performed: pending requests are aborted, global events are removed, and so on.

In reality, the lifecycle contains more stages, but for a basic implementation, these three are usually sufficient.

Implementing the Lifecycle

Let's start with the signal about the component being added to the DOM tree.

We will create a private method `mountComponent`, whose call will be placed at the end of the render method. In it, we can implement logic common to all components — for example, event handling.

We also need to support additional logic from the subclass. For this, we will create another protected method `componentDidMount`, which will be called inside `mountComponent`. This method can be overridden in the subclass:

```
/** Method for overriding in the subclass */
protected componentDidMount() {
  /** Nothing here in the base class */
}

/** Method for common mount logic and calling componentDidMount */
private mountComponent() {
  /** Common logic for all components can be implemented here */
  /** Calling the method that may have been overridden in the subclass */
  this.componentDidMount();
}

protected render() {
  const fragment = this.compile();
  if (this.domElement && fragment) {
    this.domElement.replaceWith(fragment);
  }
  this.domElement = fragment;
  this.mountComponent();
}
```

It is worth noting that the `componentDidMount` signal should fire after the component has appeared in the DOM tree. In the current implementation, this method is called immediately after creating the component, while it is still in memory.

Since on the first render the element will be added to the DOM tree later, it will not have a display context: calculations related to styles or element geometry may give inaccurate results.

Without complicating the architecture, there is no way to "catch" the moment the component is added to the DOM tree, so we will keep the current approach.

Continuing the Lifecycle Implementation

Now let's add a signal that the element is about to be replaced in the DOM tree with a new one.

We will create a private method `unmountComponent`, whose call will be placed at the very beginning of the render method. This is necessary because each time render is called, the current element is replaced with a new one — the old component is unmounted. Inside `unmountComponent`, we will add a check for the existence of the element to skip the first render.

By analogy with the previous lifecycle method, we will create a protected method `componentWillUnmount` — it will allow adding logic in the subclass:

```
/** Method for overriding in the subclass */
protected componentWillUnmount() {
  /** Nothing here in the base class */
}

/** Method for common unmount logic and calling componentWillUnmount */
private unmountComponent() {
  /** Check for element existence, needed for the first render */
  if (this.domElement) {
    /** Calling the method that may have been overridden in the subclass */
    this.componentWillUnmount();
    /** Common logic for all components can be implemented here */
  }
}

protected render() {
  this.unmountComponent();
  const fragment = this.compile();
  if (this.domElement && fragment) {
    this.domElement.replaceWith(fragment);
  }
  this.domElement = fragment;
  this.mountComponent();
}
```

Let's return to our form and add `componentDidMount` signal handling:

```
export default class Form extends Block {
  componentDidMount() {
    setTimeout(() => this.setProps({ buttonName: "Click in 3 seconds!" }), 3000);
  }
}
```

The full project code after all changes is provided in the final code listing at the end of this section.

"React on a shoestring" is taking shape: it is already reactive and even has its own lifecycle. To fully unlock its potential, a few more iterations are needed. For example, updating props during mounting is rather an artificial example of lifecycle usage. In the next lesson, you will learn how to use it properly.

Control Elements

Our framework is growing, but it still lacks maturity. For example, we have not learned how to work with events.

Event Handling

To create interactive elements, we need to be able to handle events. Buttons need to be clickable, and forms need to be validated and submitted somewhere.

Let's add a special property to store a list of events:

```
type EventListType = Partial<Record<keyof HTMLElementEventMap, (e: Event) => void>>;

/** In this object, the key is the event name and the value is the handler */
protected events: EventListType = {};
```

In this object, the key will be the event name, and the value will be the event handler. `HTMLElementEventMap` is a TypeScript interface for typing DOM events. It provides compile-time checking of event names and correct types for each event: `MouseEvent` for click, `KeyboardEvent` for keydown, and so on.

Let's create a method that will add events to the root element:

```
/** Method for adding event handlers to the element */
private attachListeners() {
  for (const eventName in this.events) {
    const eventCallback = this.events[eventName];
    if (typeof eventCallback == "function" && this.domElement) {
      this.domElement.addEventListener(eventName, eventCallback);
    }
  }
}
```

Now we need to add `attachListeners` to the lifecycle method `mountComponent`, because this relates to the internal implementation of the class:

```
/** Method that attaches handlers and calls componentDidMount */
private mountComponent() {
  this.attachListeners();
  this.componentDidMount();
}
```

Now, each time render is called, event handlers will be added to the element. This can potentially lead to memory leaks, because events will be added each time.

To protect against this, let's introduce a method that will remove added events:

```
/** Method that removes previously added events */
private removeListeners() {
  for (const eventName in this.events) {
    const eventCallback = this.events[eventName];
    if (typeof eventCallback === "function" && this.domElement) {
      this.domElement.removeEventListener(eventName, eventCallback);
    }
  }
}
```

Let's add its call to the corresponding lifecycle method:

```
/** Method that calls componentWillUnmount and detaches events */
private unmountComponent() {
  if (this.domElement) {
    this.componentWillUnmount();
    this.removeListeners();
  }
}
```

In the current architecture, it is not strictly necessary to remove event handlers on unmounting — each time the render method is called, the element is fully replaced. In a more advanced implementation, removing handlers will become a necessity.

Element References

Sometimes you need to access an element directly in the DOM tree — for example, to get a value from an input field.

For this, you could add unique data attributes and search for the desired element after mounting. To avoid doing this every time, it is convenient to store references to the desired elements in the class instance.

Let's add a special ref attribute to the template — we will store the element name in it. For example, for our form with two input fields and a button, it would look like this:

```
<form>
  <input type="text" placeholder="Login" ref="login">
  <input type="password" placeholder="Password" ref="password">
  <button>Submit</button>
</form>
```

For storing references, let's add a special refs property to the base class. Then we update the compile method, adding ref attribute processing:

```

protected refs: Record<string, Element> = {};

private compile() {
  const html = Handlebars.compile(this.template)(this.props);
  const templateElement = document.createElement("template");
  templateElement.innerHTML = html;
  const fragment = templateElement.content;

  /** Initialize the list of element references */
  this.refs = Array.from(fragment.querySelectorAll("[ref]")).reduce(
    (list, element) => {
      const key = element.getAttribute("ref");
      list[key] = element;
      element.removeAttribute("ref");
      return list;
    },
    {}
  );

  return templateElement.content.firstElementChild;
}

```

Now during template compilation, references to elements will be stored in the instance and can be accessed directly.

Let's teach our form to output filled data to the console:

```

class Form extends Block {
  protected template = `
    <form>
      <input type="text" placeholder="Login" ref="login">
      <input type="password" placeholder="Password" ref="password">
      <button>Submit</button>
    </form>
  `;

  protected events = {
    submit: (event) => {
      event.preventDefault();
      console.log(this.refs.login.value);
      console.log(this.refs.password.value);
    },
  };
}

```

No changes are needed in the main.ts file.

Try entering data in the form fields, then submit it. In the developer console, you will see the entered values. This means that:

- The submit event was bound to the root element of the form component.

- References to the input fields were saved in the refs property; their values are accessible via this.refs.login.value and this.refs.password.value.

Our first full-fledged component with event handling and element access is ready. Until now, we created components "monolithically" — for example, the entire form is described in one class. This greatly limits the capabilities of our "React on a shoestring." In real projects, we will have a set of atomic components: buttons, input fields, and many others. A true component-based approach will exist when we can use components inside other components.

In the next lesson, we will implement support for nested components in Handlebars and reassemble our form using them.

Nested Components

Once more: the component-based approach implies breaking the user interface into independent, reusable elements. Take a form: it can have any number of input fields and buttons, which are also components.

Until now, we described in the template only entities that can be compiled into DOM elements. In this lesson, you will learn how to pass real components into the template.

Handlebars does not support nested components out of the box; we will have to implement their support ourselves. We will make it so that nested components can be described as follows:

```
<header>
  {{{ Logo }}}
  {{{ SearchField }}}
  {{{ Button label="Login" }}}
</header>
```

The component names are specified inside curly braces. We use triple curly braces: in Handlebars, this syntax disables HTML escaping. This notation is called declarative.

Declarativeness is an approach to describing a task or process by result: what needs to be done, not how to do it.

In the component-based approach, declarativeness means that a component is described through its state, structure, and behavior when created — without going into implementation details. Declarativeness helps make the code cleaner and more understandable.

Component Registration

For Handlebars to work with nested components, we need to teach it to recognize and process a component. To do this, we need to extend the standard compilation scenario. This can be done using helpers.

A helper is a function that is called when a specified fragment is found in the template string. In our case, this is the component name.

We will use the `registerHelper` function: we register the component name and describe what the compiler should do when it encounters the registered name in the template string.

`registerHelper` takes two arguments: a string helper name and a handler function.

Let's create a `registerComponent` function. It will accept a component class as an argument. The component name is conveniently stored in a static class property `componentName`, so it can be used without creating an instance.

In the body of the function itself, we will need to create a class instance and add it to the template. Schematically, it looks like this:

```
import { HelperOptions } from 'handlebars';

function registerComponent(Component) {
  Handlebars.registerHelper(
    /** We take the component name from the static property */
    Component.componentName,
    function ({ hash, data }: HelperOptions) {
      /** Create an instance and return it as a string */
      const component = new Component(hash);
      return component.element().outerHTML;
    }
  );
}
```

The registered function receives an object as arguments, inside which two properties are useful:

- `hash` — an object containing the properties passed in the template string — inside curly braces.
- `data` — an object that has a root key, inside which all the data passed during instance creation is stored.

Here lies a nuance: Handlebars works only with strings. We can get the component's markup via `outerHTML`, but we will lose event handlers.

To work around this limitation in the helper, we can return a string with a placeholder in the form of a regular div with a specified data attribute. During compilation, using the data attribute, we will find the placeholder and replace it with the rendered component.

```
import { HelperOptions } from 'handlebars';

/** Unique incremental identifier for the placeholder */
let uniqueId = 0;

function registerComponent(Component) {
  Handlebars.registerHelper(
```

```

Component.componentName,
function ({ hash, data }: HelperOptions) {
  const dataAttribute = `data-component-hbs-id="${++uniqueId}"`;
  const component = new Component(hash);

  /** Mix into the component properties the __children field */
  (data.root.__children = data.root.__children || []).push({
    component,
    /** Function to replace the placeholder with the real element */
    embed(node: DocumentFragment) {
      /** Find the placeholder among children and replace it with the real
component */
      const placeholder = node.querySelector(`[${dataAttribute}]`);
      if (!placeholder) {
        throw new Error(
          `Can't find data-id for component ${Component.componentName}`
        );
      }
      const element = component.element();
      placeholder.replaceWith(element);
    },
  });

  return `

To find and replace the placeholder, a special function is needed that knows this data attribute. It is convenient to store it in the instance as a hidden property. We will add to the component properties an additional field __children (two underscores indicate that this is an internal private field), inside which the component instance and the function for inserting the real element in place of the placeholder will be stored.



The relationship between props and data.root may not be obvious, so let's elaborate. When compiling the component's template, the props object is passed to Handlebars as the root context, and a reference to it is saved in data.root. When the component registration helper executes data.root.__children = data.root.__children || [], it actually mutates the component's props, adding an array with information about child components. After compilation is complete, props.__children automatically contains all registered child components, which can then be processed in the compile() method to replace placeholders with real elements.



This is a good example of Vendor Lock — we tightly bind ourselves to the implementation of a specific tool, in this case Handlebars. In real projects, such dependencies should be minimized or at least kept under control.



## Reference to Nested Component


```

Often there is a need to access a nested component from the parent. The ref references we implemented in the previous lesson are perfect for this.

Let's add their handling to the component registration function. A component can have multiple references, so it is convenient to work with an object:

```
import { HelperOptions } from 'handlebars';

/** Unique incremental identifier for the placeholder */
let uniqueId = 0;

function registerComponent(Component) {
  Handlebars.registerHelper(
    Component.componentName,
    function (this: unknown, { hash, data }: HelperOptions) {
      const dataAttribute = `data-component-hbs-id="${++uniqueId}"`;
      const component = new Component(hash);

      /** If the component properties contain a reference, save it to the class
      property */
      if ("ref" in hash) {
        (data.root.__refs = data.root.__refs || {})[hash.ref] =
        component.element();
      }

      (data.root.__children = data.root.__children || []).push({
        component,
        embed(node: DocumentFragment) {
          const placeholder = node.querySelector(`[${dataAttribute}]`);
          if (!placeholder) {
            throw new Error(
              `Can't find data-id for component ${Component.componentName}`
            );
          }
          const element = component.element();
          placeholder.replaceWith(element);
        },
      });

      return `

## Changes in the Base Class



Let's wire all of this into the base class. Starting with the compile method.



During template compilation, we need to call the embed method for each child element in the __children array and process ref references. It is convenient to save child elements in a separate children property; it will be useful a bit later.


```

```

protected children: Block<object>[] = [];

private compile() {
  const html = Handlebars.compile(this.template)(this.props);
  const templateElement = document.createElement("template");
  templateElement.innerHTML = html;
  const fragment = templateElement.content;

  if (this.props.__children) {
    /** Save all child components */
    this.children = this.props.__children.map((child) => child.component);
    /** For each element in the array, call the embed method to replace the
    placeholder */
    this.props.__children.forEach((child) => {
      child.embed(fragment);
    });
  }

  const defaultRefs = this.props?.__refs ?? {};
  this.refs = Array.from(fragment.querySelectorAll("[ref]")).reduce(
    (list, element) => {
      const key = element.getAttribute("ref") as string;
      list[key] = element as HTMLElement;
      element.removeAttribute("ref");
      return list;
    },
    defaultRefs
  );

  return templateElement.content.firstElementChild;
}

```

Since hidden properties have appeared in the base class, we need to update its type. Let's describe the hidden properties in a separate interface and extend Props with it:

```

interface BlockOwnProps extends Object {
  __children?: Array<{
    component: Block<object>;
    embed(node: DocumentFragment): void;
  }>;
  __refs?: Record<string, Element>;
}

export abstract class Block<Props extends BlockOwnProps> {}

```

Due to the appearance of hidden properties, we need to update the setProps method. Since hidden properties are added during compilation, we need to clear them when updating external properties:

```

public setProps(props: Partial<Props>) {
  /** Reset __children and __refs to their initial values */
  this.props = { ...this.props, ...props, __children: [], __refs: {} };
}

```

```
    this.render();  
  }
```

When unmounting a component, we need to recursively call the corresponding lifecycle method on all child elements. That is why they were saved in a separate property. Let's update the `unmountComponent` method:

```
private unmountComponent() {  
  if (this.domElement) {  
    /** Call cleanup in the reverse order of creation */  
    this.children.reverse().forEach(child => child.unmountComponent());  
    this.componentWillUnmount();  
    this.removeListeners();  
  }  
}
```

Putting It All Together

The implementation is ready; all that remains is to assemble everything.

Let's create two simplified components, `Button` and `Input`:

```
import { Block } from "../block";  
  
class Button extends Block {  
  static componentName = "Button";  
  protected template = `}  
  
class Input extends Block {  
  static componentName = "Input";  
  protected template = `    ref="input">`;  
}
```

Let's register them:

```
registerComponent(Button);  
registerComponent(Input);
```

Let's take the form from the previous lesson and add our components to it:

```
class Form extends Block {  
  protected template = `  
    <form>  
      {{{ Input type="text" placeholder="Login" ref="login" }}}  
      {{{ Input type="password" placeholder="Password" ref="password" }}}  
      {{{ Button label="Login" }}}  
    </form>  
  `;  
  
  protected events = {  
    submit: (event) => {  
      event.preventDefault();  
    }  
  }  
}
```

```

        console.log(this.refs.login.value);
        console.log(this.refs.password.value);
    },
    };
}

const form = new Form();
const FormElement = form.element();
document.body.appendChild(FormElement);

```

The example is ready; let's check that it works. A form should appear on the page, and when submitted, the input field values will be printed to the console.

Now you can build complete pages from individual components. "React on a shoestring" is complete.

Final Solution — Complete Code

Below is the complete, final code for all files in the project.

Block.ts

```

import Handlebars from 'handlebars';

interface BlockOwnProps {
    __children?: Array<{
        component: Block<object>;
        embed(node: DocumentFragment): void;
    }>;
    __refs?: Record<string, Element>;
}

type EventListType = Partial<Record<keyof HTMLElementEventMap, (e: Event) => void>>;

export default abstract class Block<Props extends BlockOwnProps = BlockOwnProps> {
    protected abstract template: string;

    protected props = {} as Props;

    private domElement: Element | null = null;

    protected children: Block<object>[] = [];

    protected refs: Record<string, Element> = {};

    protected events: EventListType = {};
}

```

```

constructor(props: Props = {} as Props) {
  this.props = props;
}

public element(): Element | null {
  if (!this.domElement) {
    this.render();
  }
  return this.domElement;
}

public setProps(props: Partial<Props>) {
  this.props = { ...this.props, ...props, __children: [], __refs: {} } as
Props;
  this.render();
}

protected componentDidMount() {}

private mountComponent() {
  this.attachListeners();
  this.componentDidMount();
}

protected componentWillUnmount() {}

private unmountComponent() {
  if (this.domElement) {
    this.children.reverse().forEach(child => child.unmountComponent());
    this.componentWillUnmount();
    this.removeListeners();
  }
}

private attachListeners() {
  for (const eventName in this.events) {
    const eventCallback = this.events[eventName];
    if (typeof eventCallback == 'function' && this.domElement) {
      this.domElement.addEventListener(eventName, eventCallback);
    }
  }
}

private removeListeners() {
  for (const eventName in this.events) {
    const eventCallback = this.events[eventName];
    if (typeof eventCallback === 'function' && this.domElement) {
      this.domElement.removeEventListener(eventName, eventCallback);
    }
  }
}

protected render() {

```

```

    this.unmountComponent();
    const fragment = this.compile();
    if (this.domElement && fragment) {
        this.domElement.replaceWith(fragment);
    }
    this.domElement = fragment;
    this.mountComponent();
}

private compile(): Element | null {
    const html = Handlebars.compile(this.template)(this.props);
    const templateElement = document.createElement('template');
    templateElement.innerHTML = html;
    const fragment = templateElement.content;

    if (this.props.__children) {
        this.children = this.props.__children.map((child) => child.component);
        this.props.__children.forEach((child) => {
            child.embed(fragment);
        });
    }

    const defaultRefs = this.props?.__refs ?? {};
    this.refs = Array.from(fragment.querySelectorAll('[ref]')).reduce(
        (list, element) => {
            const key = element.getAttribute('ref') as string;
            list[key] = element as HTMLElement;
            element.removeAttribute('ref');
            return list;
        },
        defaultRefs,
    );

    return templateElement.content.firstElementChild;
}
}

```

ComponentRegistry.ts

```

import Handlebars from 'handlebars';
import { HelperOptions } from 'handlebars';

let uniqueId = 0;

function registerComponent(Component: any) {
    const dataAttribute = `data-component-hbs-id="${++uniqueId}"`;

    Handlebars.registerHelper(
        Component.componentName,
        function (this: unknown, { hash, data }: HelperOptions) {
            const component = new Component(hash);

```

```

    if ('ref' in hash) {
      (data.root.__refs = data.root.__refs || {})[hash.ref] =
component.element();
    }

    (data.root.__children = data.root.__children || []).push({
      component,
      embed(node: DocumentFragment) {
        const placeholder = node.querySelector(`[${dataAttribute}]`);
        if (!placeholder) {
          throw new Error(`Can't find data-id for component
${Component.componentName}`);
        }

        const element = component.element();
        if (!element) {
          throw new Error('Component element is not created');
        }

        placeholder.replaceWith(element);
      }
    });

    return `<div ${dataAttribute}></div>`;
  }
);
}

export { registerComponent };

```

Form.ts

```

import Block from './Block'

export default class Form extends Block {
  protected template = `
    <form>
      {{{ Input type="text" placeholder="Login" ref="login" }}}
      {{{ Input type="password" placeholder="Password" ref="password" }}}
      {{{ Button label="Login" }}}
    </form>
  `;

  protected events = {
    submit: (event) => {
      event.preventDefault();
      console.log(this.refs.login.value);
      console.log(this.refs.password.value);
    },
  };

  componentDidMount() {

```

```
    setTimeout(() => this.setProps({ buttonName: "Click in 3 seconds!" }), 3000);
  }
}
```

main.ts

```
import Form from './Form';
import { registerComponent } from './ComponentRegistry';
import Block from './Block';

class Button extends Block {
  static componentName = 'Button';

  protected template = `
    <button>{{label}}</button>
  `;
}

class Input extends Block {
  static componentName = 'Input';

  protected template = `
    <input type="{{type}}" placeholder="{{placeholder}}" ref="{{ref}}">
  `;
}

registerComponent(Button);
registerComponent(Input);

const form = new Form({ buttonName: "Initial state" });
const FormElement = form.element();
document.body.appendChild(FormElement);
form.setProps({ buttonName: "Click!" });
```